SQL Set Operations



Set Operations (~)

- If you do not care about the order, then could be useful to adopt set operations.
- **UNION**
- **INTERSECT**
- **EXCEPT**

Set Operations (~)

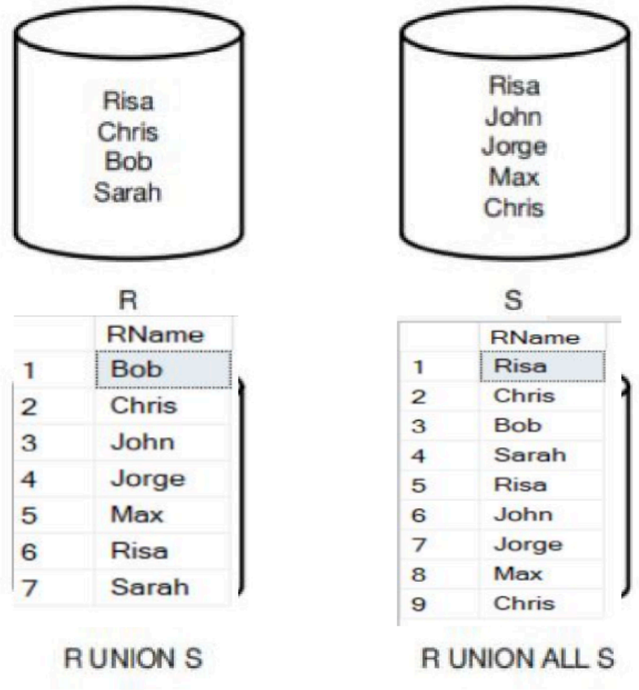
- **UNION/UNION ALL**

```
-- union
SELECT * FROM R
UNION
SELECT * FROM S
```

```
--union all
SELECT * FROM R
UNION ALL
SELECT * FROM S
```

- UNION combine the result sets of **2 or more SELECT statements.**
  - UNION ALL: does **not remove** duplicate rows.
  - UNION: **remove duplicate** rows.

- UNION All often have better performance than UNION.

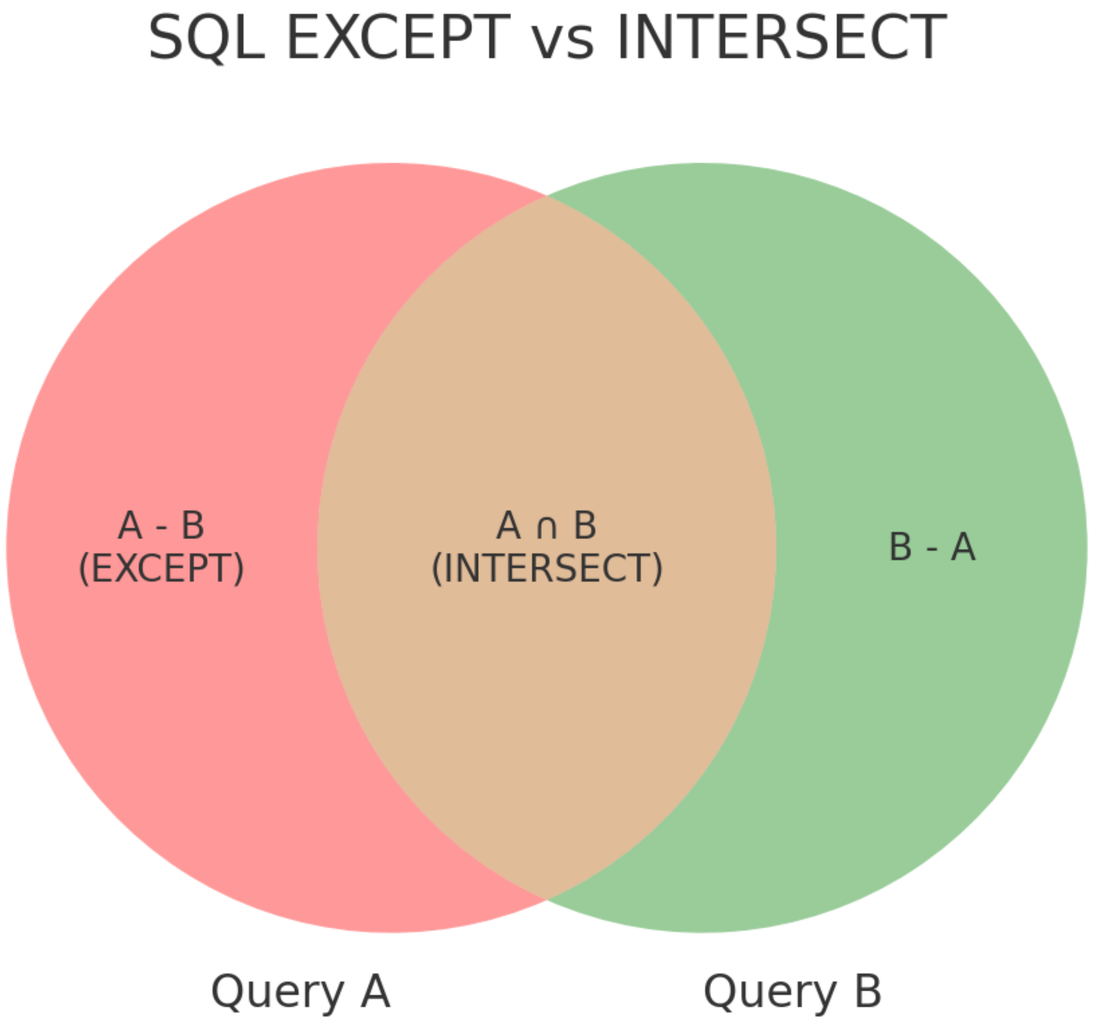


Set Operations (~)

- **EXCEPT/INTERSECT**

  - **EXCEPT**: returns rows from the first query that do not appear in the second query.
  - (will return row if ANY of the field value is different)

  - **INTERSECT** : returns common records of the two SELECT statements.



Intersect

- MySQL doesn’t support intersection directly, but INNER JOIN can be used to Implement intersection

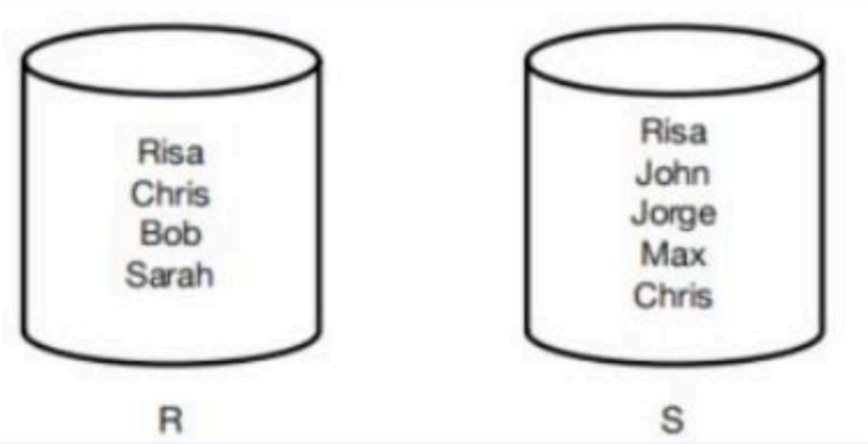
```
SELECT R.RName
FROM R
INNER JOIN S ON R.RName = S.SName;
```

Results | Messages

	RName
1	Chris
2	Risa

Difference (Except)

- Display the values in the first select statement **MINUS** any values found in the second select statement




```
SELECT * FROM R
EXCEPT
SELECT * FROM S
```

Results Messages

	RName
1	Bob
2	Sarah

Index

Index

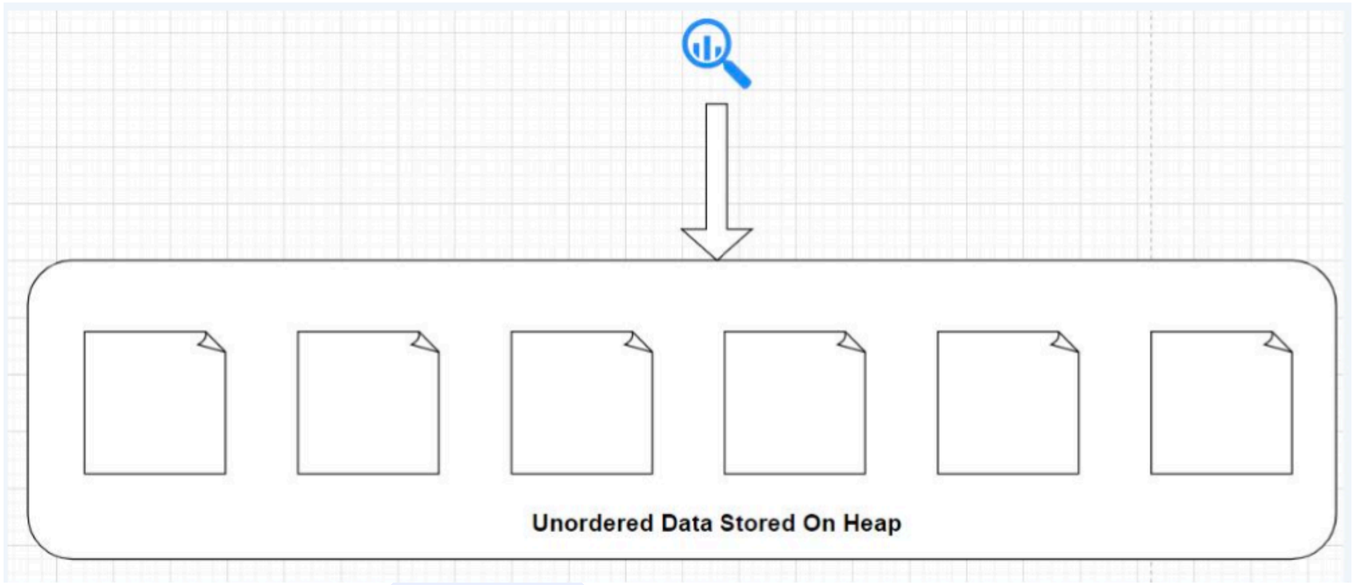
- Indexes are used to **speed up** the retrieval of data from a database by **providing quick access paths to the data**
- Operate similar to index in a book
- Commonly implemented using a B+ Tree(some DBMS use B-Tree)
- **Clustered Index** is (usually) associated with **PK**, it **physically orders the rows** in the table
- **Non-clustered Index** is (usually) associated with **Unique Keys**, it stores **data pointer** to the data in the table
- Tables without a Clustered Index are called **HEAP Tables**
- Keys ≠ Indexes
- Purpose: Indexes make **querying and scanning faster** but may **slow down write operations** INSERT, UPDATE, and DELETE b/c index also has to be updated.

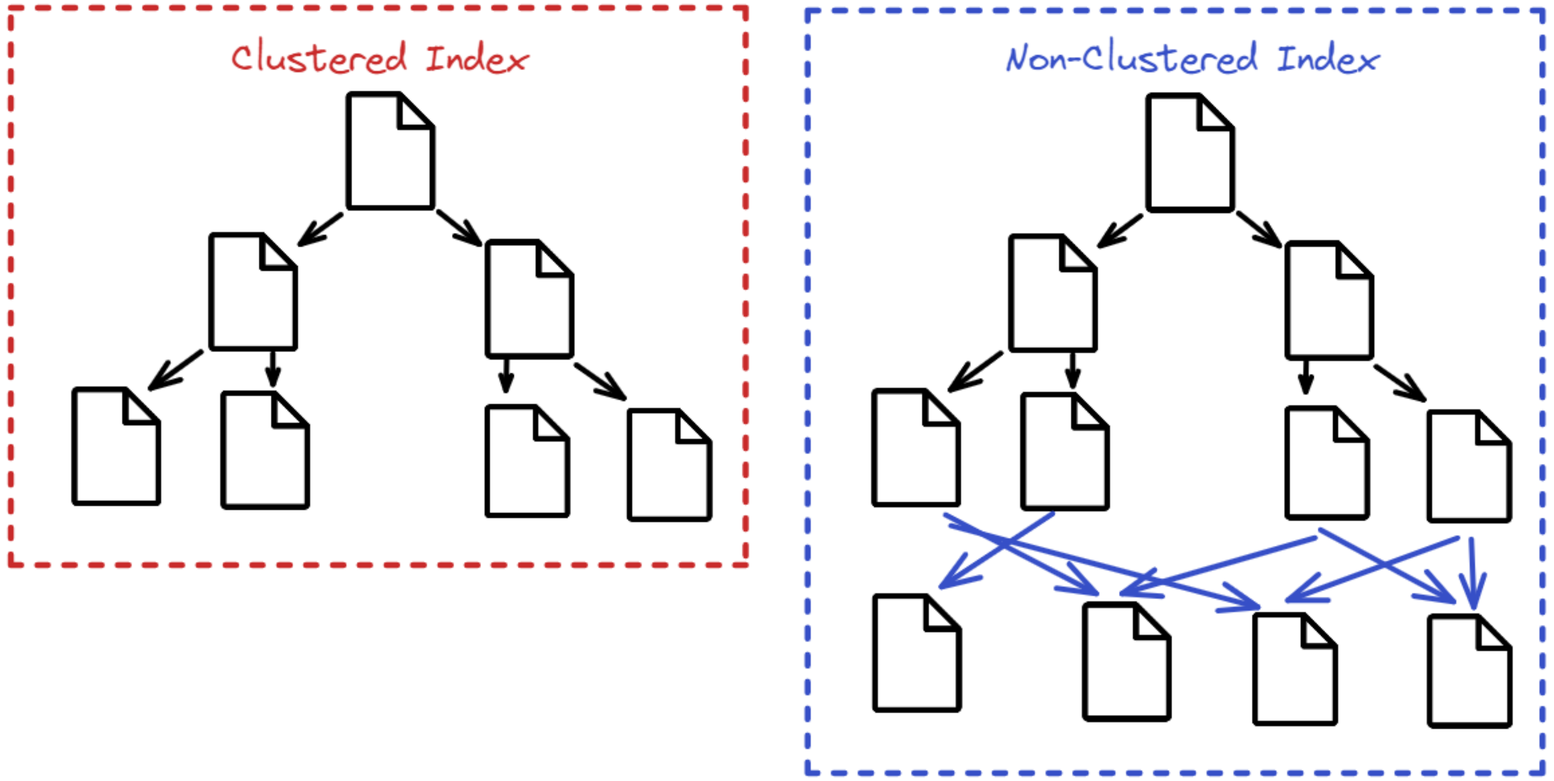
Consider a table `Employees` with `EmployeeID` as an indexed column. The index speeds up queries that search for a specific `EmployeeID`.

```
SELECT * FROM Employees WHERE EmployeeID = 101;
```

No Index Table

- If a table has **no index** at all, when new data is inserted, they are added wherever there is **free space**, and in **no particular order.**
- If you want to look up some data, you need to scan the entire table





- A **clustered index** organizes and stores the actual data rows in the table in the order of the index key and is best suited for **range queries**. Default on PK. One per table.(WHERE id BETWEEN 100 AND 200)
- A **non-clustered index** creates a separate structure with **pointers** to the data rows and is useful for quick lookups on non-primary key columns or frequently queried columns.

- **Clustered vs Non-clustered Index**
- Think of a physical library where books are the data records in your database.
- A **clustered index** is like how books are **physically arranged** on the shelves by their call numbers. Each book has exactly one physical location, and all books are sorted in a specific order. When you walk down the shelf, you're walking through the data in the order defined by the clustered index. Just like a library can only organize its physical books in one way on the shelves, a table **can only have one clustered index**.
- A **non-clustered index** is like the library's card catalog system. You can have multiple card catalogs – one sorted by author names, another by titles, another by subjects, etc. Each card contains the book's call number, telling you where to find the actual book on the shelf. Similarly, non-clustered indexes contain **pointers to the actual data rows**, and you can have **multiple non-clustered indexes on a table**, each organizing the data differently.
- So when you search:
  - With **a clustered index**, you're directly walking to the shelf and finding books in their physical order (faster for range queries)
  - With **a non-clustered index**, you first look up the location in the card catalog, then use that information to go find the actual book (requires an extra step but gives you more ways to find the data)

Index choice?

- Too many index will impede queries performance.
- Can be applied to Views as well.
- We can consider the following when picking a column to be our index:
  - **Query Patterns**
    - Frequent WHERE clauses
    - Common JOIN conditions
    - GROUP BY columns
    - ORDER BY columns
  - **Column Properties**
    - High selectivity (many unique values)
    - Low-medium cardinality(not too few unique value)
    - Relatively static values(value don’t change often)
    - Not frequently updated(index maintenance overhead)

Views and Materialized Views



View

- **What** is a View?
- A view is often seen as a virtual table
- It is updated every time is referred to.
- It displays data that you choose, but does not necessarily hold any data
- Good for security since you can prevent showing extra data
- DML operations just happen on the table:
  - You can modify data on view level, and the source data will be updated as well.

View

- **Why View?**
- A well-designed database **only provides queries through an interface** in the form of views and does not allow users or client applications to query the tables in the schema directly.
- Thus, views **provide an abstraction** of the actual schema that ensures that **schema changes do not affect SQL queries**. If the name of a table or the definitions of columns changes, then the database architect updates the view definition and client and user queries against the view continue to function without modification.
- In addition, views also help enforce **security** as the data can only be accessed through views and the data in the actual tables is hidden from all users.
- Essentially:
  - **Security**
  - provides a level of **abstraction** to accommodate schema changes.

View

- **Create** a View
- Let’s say that email addresses in the Author table of some marketing databases **should not be accessible** for non-marketing personnel but should at least be partially available for matching. So, rather than providing access to the data in the Author table, we define a view that can be use by non-marketing person.
- The view is defined below as the result of a SQL query. Notice the renaming of columns from the original table.

```
CREATE VIEW AuthorNM
 (fullname, email)
 AS
 SELECT name,
        substr(email,1,2) || '****' || substr(email,-5)
   FROM Author;
```

Jo****@mail.com

Note that SQLite uses || for string concatenation while MySQL uses the concat() function. PostgreSQL can use both.

View Conclude

- **View vs Table**
- A view is a **virtual table** defined by a SQL SELECT statement and behaves in queries exactly like a table. The content of the view is materialized generally **only when the view is involved in a query**, although databases may cache views to improve performance. Since a view is not stored, it do not consume storage; only the view definition is stored in the database.

View

- **Materialized View**
  - Persists the data returned from the view query
  - Refresh data and get updated as data changes in the underlying tables.
    - Better performance
    - Costly
  - Refresh manually, even concurrently (if supported by the RDBMS)

```sql
CREATE MATERIALIZED VIEW monthly_sales
AS
SELECT
    product_id,
    product_name,
    SUM(sales_amount) as total_sales,
    DATE_TRUNC('month', sale_date) as sale_month
FROM sales
GROUP BY
    product_id,
    product_name,
    DATE_TRUNC('month', sale_date);
```

View or Materialized View?

- Use a **Standard View** when..
  - **Real-Time**, Always Up-to-Date Data: If you need your query to reflect the latest data **at all times**, a standard view might be preferable.
  - **Infrequent Queries**: If the underlying query is simple and the view is queried infrequently, save some additional storage cost.
- Use a Materialized View when..
  - **Frequent Queries** on Large Data Sets: frequently run the same query on large datasets and the query is computationally expensive (e.g., lots of joins, aggregations)
  - **Low Latency Requirements**: If you need near-instant query responses on pre-aggregated or pre-joined data, materialized views offer much faster results.

Stored Procedures and Triggers



Stored Procedure

- A stored procedure is a **set of SQL statements** that are **pre-compiled** and stored in the database.
  - To create a stored procedure you need to use CREATE PROCEDURE statement.
- It accepts input and output parameters (not mandatory).
  - Parameter declaration: **IN**, **OUT**..
- It executes the SQL statements, and returns a result set if any
  - To execute a stored procedure you need to use **CALL** statement.

- Benefits of SP:
  - Performance Improvement
  - Code Reusability

Stored Procedure

- Parameters we can use in SP:
  - Input Parameters:
    - Used to **pass values into** the procedure, like a argument for a function

```
CREATE PROCEDURE example(IN emp_id INT)
```

  - Output Parameters:
    - Used to **return values** from the procedure
    - Can return multiple values using multiple **OUT** parameters

```
CREATE PROCEDURE example(IN emp_id INT, OUT emp_name TEXT)
```

- Default Parameter
  - Assigns the parameter with a default value.
  - If the caller doesn’t pass that argument, the default will be used

```
CREATE PROCEDURE example(IN bonus_rate NUMERIC DEFAULT 0.1)
```

Stored Procedure

```
DELIMITER //

CREATE PROCEDURE GetOfficeByCountry(
        IN countryName VARCHAR(255)
)
BEGIN
        SELECT *
        FROM offices
        WHERE country = countryName;
END //

DELIMITER ;
```

```
CALL GetOfficeByCountry('USA');
```

officeCode	city	phone	addressLine1	addressLine2	state	country	postalCode	territory
1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA	USA	94080	NA
2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA	USA	02107	NA
3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY	USA	10022	NA

Stored Procedure

```
DELIMITER $$

CREATE PROCEDURE GetOrderCountByStatus (
        IN  orderStatus VARCHAR(25),
        OUT total INT
)
BEGIN
        SELECT COUNT(orderNumber)
        INTO total
        FROM orders
        WHERE status = orderStatus;
END$$

DELIMITER ;
```

```
CALL GetOrderCountByStatus('Shipped',@total);

SELECT @total;
```

	@total
▶	303

Dynamic SQL

- Dynamic SQL is a programming technique where SQL **statements are constructed at runtime** rather than being written as static statements.
- SQL Server

```
-- Static SQL
SELECT * FROM Customers WHERE Country = 'USA';

-- Dynamic SQL equivalent
DECLARE @country VARCHAR(50) = 'USA';
DECLARE @sql NVARCHAR(MAX) = 'SELECT * FROM Customers WHERE Country = @countryParam
EXEC sp_executesql @sql, N'@countryParam VARCHAR(50)', @country;
```

- In MySQL:

```
SET @sql = 'SELECT * FROM Customers WHERE Country = ?';
PREPARE stmt FROM @sql;
SET @country = 'USA';
EXECUTE stmt USING @country;
DEALLOCATE PREPARE stmt;
```

Dynamic SQL

```
-- Imagine building a search function that handles
multiple optional filters(SQL Server)

DECLARE @sql NVARCHAR(MAX);
DECLARE @name VARCHAR(50) = 'John';
DECLARE @minAge INT = 25;
DECLARE @city VARCHAR(50) = NULL;
```

```
-- Build the query dynamically based on provided filters
SET @sql = 'SELECT * FROM Customers WHERE 1=1 ';

-- Only add conditions for non-null parameters
IF @name IS NOT NULL
    SET @sql += 'AND Name LIKE ''%'' + @nameParam + ''%'' ';
IF @minAge > 0
    SET @sql += 'AND Age >= @ageParam ';
IF @city IS NOT NULL
    SET @sql += 'AND City = @cityParam';

-- Execute with parameters
EXEC sp_executesql @sql,
    N'@nameParam VARCHAR(50), @ageParam INT, @cityParam VARCHAR(50)',
    @name, @minAge, @city;
```

Trigger

- A **trigger** is a named database object that's **associated with a table** and **automatically executes** when a certain event happens: (INSERT, UPDATE, DELETE) occurs for the table.
- Components:
  - **Event**: The type of action (INSERT, UPDATE, DELETE) that will activate the trigger.
  - **Timing**: Specifies when the trigger will be executed: BEFORE or AFTER the event.
  - **Triggered Action**: The set of SQL statements to execute when the trigger activates.
- Triggers are often used to:
  - **Ensure data integrity** rules beyond the standard constraints.
  - Automatically format or transform data.
  - ...

*SNOWFLAKE does not support triggers directly, but we can simulate with a **stream** and **task**(change tracking + scheduled jobs).

Triggers - Insert

```sql
CREATE TRIGGER check_salary_before_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END;
```

Before Insert Trigger (validation)

```sql
CREATE TRIGGER audit_insert_employee
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit (employee_id, action, action_date)
    VALUES (NEW.id, 'INSERT', NOW());
END;
```

After Insert Trigger (audit)

Triggers - Update/ Delete

```
CREATE TRIGGER format_email_before_update
BEFORE UPDATE ON customers
FOR EACH ROW
BEGIN
    SET NEW.email = LOWER(NEW.email);
    SET NEW.last_modified = NOW();
END;
```

Before Update Trigger (data modification)

```
CREATE TRIGGER backup_deleted_records
AFTER DELETE ON orders
FOR EACH ROW
BEGIN
    INSERT INTO orders_backup
    SELECT *, NOW(), USER()
    FROM orders
    WHERE id = OLD.id;
END;
```

After Delete Trigger (backup record)

Triggers - Great flexibility

```
CREATE TRIGGER maintain_product_inventory
AFTER INSERT OR UPDATE OR DELETE ON order_items
FOR EACH ROW
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE products
        SET stock = stock - NEW.quantity
        WHERE id = NEW.product_id;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE products
        SET stock = stock + OLD.quantity
        WHERE id = OLD.product_id;
    ELSIF TG_OP = 'UPDATE' THEN
        UPDATE products
        SET stock = stock + OLD.quantity - NEW.quantity
        WHERE id = NEW.product_id;
    END IF;
END;
```

Can combine multiple events together

```
CREATE TRIGGER check_order_status
BEFORE UPDATE ON orders
FOR EACH ROW
BEGIN
    IF OLD.status = 'COMPLETED' AND NEW.status != 'COMPLETED' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot change status of completed order';
    END IF;

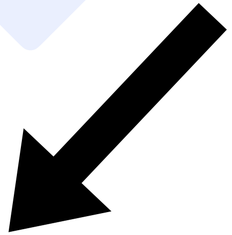
    IF NEW.status = 'SHIPPED' AND NEW.shipping_date IS NULL THEN
        SET NEW.shipping_date = CURRENT_DATE();
    END IF;
END;
```

Customized Order status logic

Triggers: FOR EACH ROW vs FOR EACH STATEMENT

```
INSERT INTO employees (name) VALUES ('John'), ('Jane'), ('Bob');
```

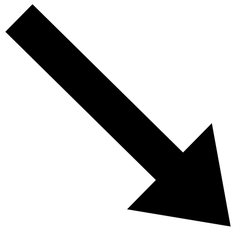
Trigger _ times



```
CREATE TRIGGER example_trigger
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    -- Executes once for each row affected
    INSERT INTO audit_log VALUES (NEW.id, NEW.name);
END;
```

This is a row-level trigger that executes once for every row affected by the triggering statement.

Trigger _ times



```
-- PostgreSQL syntax
CREATE TRIGGER example_statement_trigger
AFTER INSERT ON employees
FOR EACH STATEMENT
EXECUTE FUNCTION log_batch_insert();
```

This executes once per SQL statement, regardless of how many rows are affected.

*MySQL: Only supports FOR EACH ROW

SQL Query Optimization

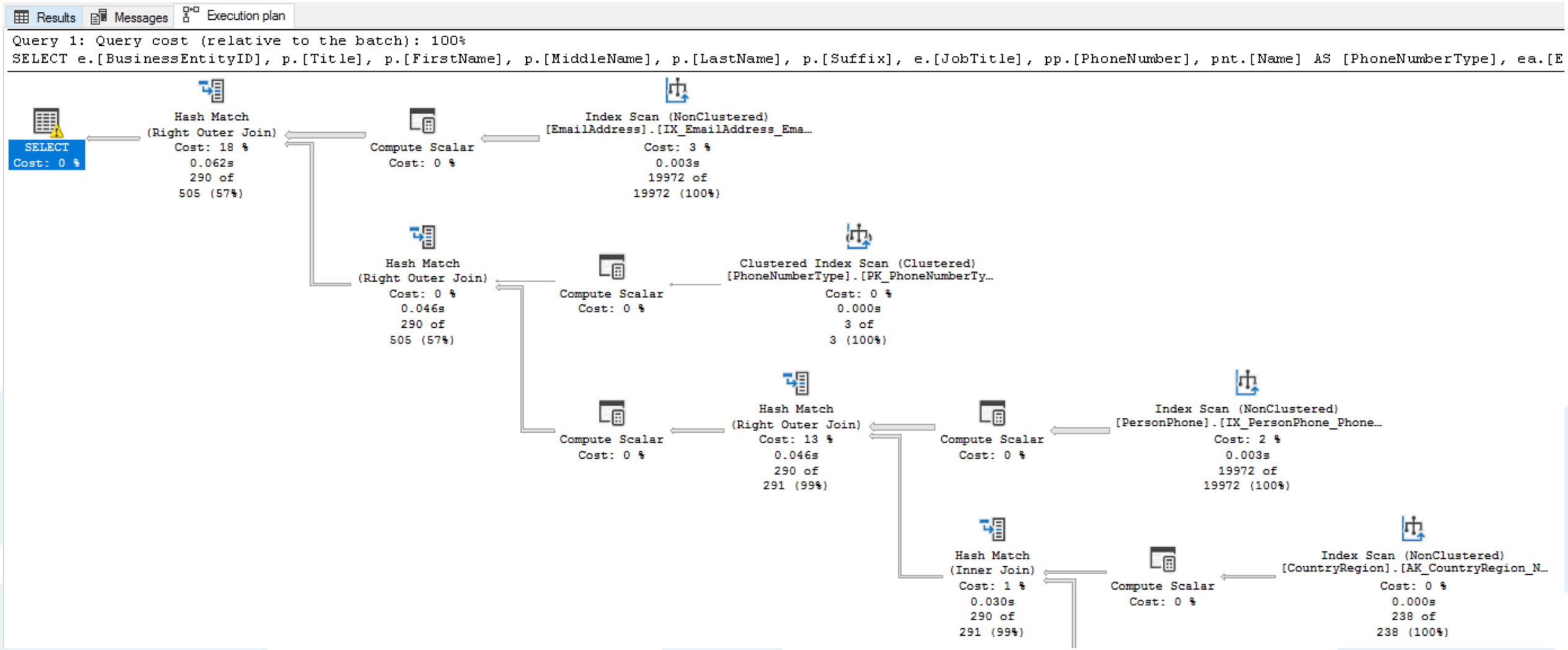


Query Optimization

- Recall the query logical processing order ...
  - **FROM** – Specifies the tables involved.
  - **WHERE** – Filters rows based on a condition.
  - **GROUP BY** – Groups rows sharing values of certain columns.
  - **HAVING** – Filters groups created by GROUP BY.
  - **SELECT** – Chooses columns to display in the result.
  - **DISTINCT** – Removes duplicate rows.
  - **ORDER BY** – Sorts the results.
  - **LIMIT / OFFSET** – Restricts the number of rows returned (optional).

Query Optimization

- Examine the **query execution plan**: map of how the database will run your query.
- For instance in Snowflake, the key components in a plan are:
  - **Operations**: the actions performed (e.g., table scans, joins).
    - **Scan**: Reads data from the table (full = slow or filtered scan = faster).
    - **Join**: Combines rows from multiple tables.
    - **Aggregate**: Performs operations like SUM, AVG, etc.
    - **Filter**: Applies conditions to limit data.
    - **Sort**: Arranges results.
  - **Partitions**: Snowflake divides data into micro-partitions/ Queries can process partitions in parallel
  - **Rowset Processing**: Shows data movement between steps(like pipeline flow).
  - **Output Columns**: Columns involved in each step.



In MS SQL Server

	step	id	parentOperators	operation	objects	alias	expressions	partitionsTotal	partitionsAssigned	bytesAssigned
1	null	null	null	GlobalStats	null	null	null	5	5	11264
2	1	0	null	Result	null	null	C.CUSTOMER_NAME, COUNT(S.SALE_ID), RANK() OVER (ORDER BY COUNT(S.SAL	null	null	null
3	1	1	[0]	Sort	null	null	RANK() OVER (ORDER BY COUNT(S.SALE_ID) DESC NULLS FIRST) ASC NULLS LAS	null	null	null
4	1	2	[1]	WindowFunction	null	null	SUM(S.QUANTITY) OVER (PARTITION BY S.CUSTOMER_ID ORDER BY S.SALE_DATE	null	null	null
5	1	3	[2]	InnerJoin	null	null	joinKey: (S.CUSTOMER_ID = C.CUSTOMER_ID)	null	null	null
6	1	4	[3]	TableScan	PLAYGRC	S	CUSTOMER_ID, SALE_DATE, QUANTITY, TOTAL_AMOUNT	1	1	2560
7	1	5	[3]	LeftOuterJoin	null	null	joinKey: (C.CUSTOMER_ID = SYS_VW.CUSTOMER_ID_0)	null	null	null
8	1	6	[5]	JoinFilter	null	null	joinKey: (S.CUSTOMER_ID = C.CUSTOMER_ID)	null	null	null
9	1	7	[6]	WindowFunction	null	null	RANK() OVER (ORDER BY COUNT(S.SALE_ID) DESC NULLS FIRST)	null	null	null
10	1	8	[7]	InnerJoin	null	null	joinKey: (S.CUSTOMER_ID = C.CUSTOMER_ID)	null	null	null
11	1	9	[8]	Filter	null	null	(COUNT(S.SALE_ID)) > 3	null	null	null
12	1	10	[9]	Aggregate	null	null	aggExprs: [COUNT(S.SALE_ID)], groupKeys: [S.CUSTOMER_ID]	null	null	null
13	1	11	[10]	TableScan	PLAYGRC	S	SALE_ID, CUSTOMER_ID	1	1	2560
14	1	12	[8]	JoinFilter	null	null	joinKey: (S.CUSTOMER_ID = C.CUSTOMER_ID)	null	null	null
15	1	13	[12]	TableScan	PLAYGRC	C	CUSTOMER_ID, CUSTOMER_NAME	1	1	2048
16	1	14	[5]	Filter	null	null	ROW_NUMBER() OVER (PARTITION BY S.CUSTOMER_ID ORDER BY S.SALE_DATE A	null	null	null
17	1	15	[14]	WindowFunction	null	null	ROW_NUMBER() OVER (PARTITION BY S.CUSTOMER_ID ORDER BY S.SALE_DATE A	null	null	null
18	1	16	[15]	InnerJoin	null	null	joinKey: (P.PRODUCT_ID = S.PRODUCT_ID)	null	null	null
19	1	17	[16]	TableScan	PLAYGRC	P	PRODUCT_ID, PRODUCT_NAME	1	1	1536
20	1	18	[16]	Filter	null	null	S.CUSTOMER_ID IS NOT NULL	null	null	null
21	1	19	[18]	JoinFilter	null	null	joinKey: (S.CUSTOMER_ID = C.CUSTOMER_ID)	null		
22	1	20	[19]	TableScan	PLAYGRC	S	CUSTOMER_ID, PRODUCT_ID, SALE_DATE	1	1	2560

Ask Copilot

In Snowflake

Query Optimization

- **Create proper index**
- Indexes are a key performance booster when performing filtering, joining and ordering.
  - Any index added on a table **slows down write operations**.
  - Moreover, indexes occupy space on disk and require maintenance. Need to **update the index periodically** with the help of system table PostgreSQL pg_stat_user_indexes or the MySQL table_io_waits_summary_by_index_usage
- Syntax for creating index varies among different RDBMS

```
CREATE INDEX index_name
ON table_name (column1, column2, ...);

DROP INDEX table_name.index_name;
```

Generic SQL

```
ALTER TABLE <name> CLUSTER BY ( <expr1> [ , <expr2> ... ] )

ALTER TABLE <name> DROP CLUSTERING KEY
```

Snowflake

Query Optimization

- In Snowflake, there is no concept of traditional indexes (like in other relational databases such as MySQL or PostgreSQL) because Snowflake automatically optimizes data storage and retrieval through techniques such as **micro-partitioning**, **clustering**, and the use of **result caching**.

- More soon...

Query Optimization

- **Optimize Join** (Avoid Full Table Scan)
- Inner Join should be preferred

```
SELECT *
FROM ORDERS LEFT JOIN USERS ON ORDERS.NAME = USERS.NAME
WHERE USERS.NAME IS NOT NULL
```

- Avoid functions in Joins (use the same column data types for joining):

```
SELECT *
FROM users
JOIN orders ON UPPER(users.user_name) = UPPER(orders.user_name)
```

- Think about if join is necessary..

```
SELECT d.DepartmentName, e.Name, e.Salary
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE e.Salary = (
  SELECT MAX(Salary)
  FROM Employees e2
  WHERE e2.DepartmentID = e.DepartmentID
);
```

```
SELECT Name, DepartmentID, Salary
FROM (
  SELECT Name, DepartmentID, Salary,
         RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) AS rank
  FROM Employees
) e
WHERE rank = 1;
```

Query Optimization

- **Optimize Filtering**
- **Avoid function usage in where**

```
SELECT count(*)
FROM SALES
WHERE CAST(sale_date AS DATE) > '2024-10-07';
```

- **Only SELECT the necessary columns (avoid SELECT *)**
- **Use LIMIT and OFFSET to do paginating (select part of the full table)**
- **Filter early (filter in WHERE rather than in HAVING)**

```
SELECT session_date, count(user_id) nr_sessions
FROM sessions
WHERE user_id is not null
GROUP BY session_date
```

```
SELECT session_date, count(user_id) nr_sessions
FROM sessions
GROUP BY session_date
HAVING count(user_id) > 0;
```

Query Optimization

- **Avoid DISTINCT** (ask yourself if DISTINCT is really necessary here)
- Use a **materialized view**
- Table **partitioning**
- Increase computational capacity (**scale up**)
- Understand the business logic so that we can change the underlying **data model**
- **Big Data Tool** Spark/Hadoop/...
- ...

How to optimize?

- You have a query that fetches data about listing, bookings and reviews. But it’s taking too long. How to optimize?

```sql
SELECT l.listing_id, l.listing_name, b.booking_amount, r.review_text
FROM Listing l
JOIN Bookings b ON l.listing_id = b.listing_id
JOIN Reviews r ON l.listing_id = r.listing_id
WHERE l.status = 'available'
AND CAST(b.check_in_date,DATE) >= '2024-01-01';
```

```sql
SELECT l.listing_id, l.listing_name, b.booking_amount, r.review_text
FROM Listing l
JOIN Bookings b
ON l.listing_id = b.listing_id
 AND b.check_in_date >= '2024-01-01'           --This reduces the number of rows processed in the Join
JOIN Reviews r ON l.listing_id = r.listing_id
WHERE l.status = 'available';
```